Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 233 (2024) 919–928

Procedia Computer Science

www.elsevier.com/locate/procedia

5th International Conference on Innovative Data Communication Technologies and Application (ICIDCA 2024)

WebGL vs. WebGPU: A Performance Analysis for Web 3.0

SatyadhyanChickerur [a,b,*], Sankalp Balannavar[b], Pranali Hongekar[b], Aditi Prerna[b], Soumya Jituri[b]

[a] *Center for Artificial Intelligence Research, KLE Technological University, Hubballi 580031, India*
[b] *School of Computer Science and Engineering, KLE Technological University, Hubballi 580031, India*
chickerursr@kletech.ac.in, balannavar.sankalp@gmail.com, pranalihongekar99@gmail.com, aditiprerna8@gmail.com, soumyajituri1234@gmail.com

Abstract

This study investigates web 3.0 heterogeneous computing with webGL, webGPU, and IPFS. The primary focus is on the benefits of utilising these technologies to enhance the functionality and performance of web 3.0 applications. The study investigates web 3.0 as it currently exists and the constraints that developers face due to graphic, computational, and storage capabilities. According to the findings, incorporating webGL and webGPU can considerably increase user experience, speed, efficacy, and decentralization. Finally, this study summarizes the importance of continuing research in this subject, particularly with relation to platform interoperability and the future prospects of heterogeneous computing on web 3.0 via graphical APIs.

© 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application

Keywords: WebGL; WebGPU; Rendering; Performance analysis; Web3.0;

1. Introduction

The way people use internet is changing dramatically with the introduction of Web 3.0 technology [1,2,3,4,5,6,7,8]. Users now have more control over their data with the decentralization of data and applications and the growing usage of blockchain technology, altering the conventional client-server architecture. Nevertheless, Web 3.0 technology's present constraints prevent it from reaching its full potential. One such restriction is the client devices' limited graphical and processing capacity. The proposed research looks into how researchers can use WebGL, WebGPU, and IPFS can be used for heterogeneous computing on Web 3.0. The study focuses on the possible benefits of using these technologies to enhance web 3.0 apps' functionality and performance. In addition to examining Web 3.0's current state and limits, this study thoroughly examines the benefits of integrating WebGL and WebGPU. According to the study, implementing these technologies can significantly boost user experience, decentralization, speed, and efficiency. De-

1877-0509 © 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application
10.1016/j.procs.2024.03.281

velopers and academics interested in Web 3.0 technology and graphical web APIs will find this research study helpful. The significance of ongoing research in this field is emphasized, particularly on the compatibility of different technologies, and a roadmap for the advancement of heterogeneous computing on Web 3.0 is provided.

1.1. Motivation

Existing apps should convert from Web 2. x framework to Web 3.0 since Web 3.0 is the future configuration of the internet. It's common knowledge that "Web3" is a modern, enhanced internet. The core idea behind Web3 is to give consumers back control and ownership via NFTs, cryptocurrencies, and blockchains. Our primary objective is to use graphical APIs such as WebGL[9] and WebGPU[10] while developing Web 3.0 compatible applications.

1.2. Objectives

The main objectives of this research are to:

- Compare the performance of WebGL and WebGPU on different platforms.
- Host web pages on a Web 3.0 network consisting of multiple nodes.
- Optimize the applications that work on different types of nodes with different computational capabilities.
- Optimize the performance for low-power devices, which should also be able to run graphic-intensive applications on Web 3.0.

1.3. Literature Survey

The paper's author[11] claims that small and medium-sized enterprises in India have been investigating the advantages and drawbacks of integrating Web 3.0 technologies to boost productivity and efficiency. After conducting a comprehensive review of the literature, researchers created a semi-structured questionnaire with open-ended questions to get input from managers of 40 Indian SME companies that are representative of the five major economic sectors of the nation. NVivo 11, a computer-assisted qualitative data analysis application, assessed the collected data and categorized themes. The researchers utilized content analysis to investigate the data acquired from the semi-structured interviews. This study identified five significant topics and twelve supporting themes to highlight the key advantages and challenges perceived by the managerial leadership of SMEs. The five main subjects covered in this study are financial and technological issues, privacy and security, new feature development, data and services integration, and organizational obstacles. The executive leadership of SMEs will benefit from the study's findings as they plan and build their short- and long-term information systems initiatives. Additionally, they will enable SMEs' organizational leadership to optimize their information technology resources, boosting the firms' competitiveness and productivity. Because Web 3.0 technologies are still in their infancy, researchers haven't thoroughly studied developing nations like India. The authors in this paper [12]explain how Web 3.0 can benefit users and enterprises. It will change how individuals work and play and how businesses utilize data to run their operations, market, and sell their products. Web 3.0 represents a significant transition in computing operations from information-centric to knowledge-centric. Web 3.0 will enhance our experience with the Internet by enabling humans and machines to interact, modify, share, and use knowledge in new and innovative ways on a never-before-seen scale. In addition, through cost reductions, increased efficiency, increased effectiveness, and new functionalities that were not previously feasible or economically viable, semantic technologies have the potential to improve significantly life cycle economics and capabilities. This paper looks at how the semantic web and Web 3.0 can spur the creation of new business models and value. To do that, the author analyzes the role that Web 3.0 plays in business and its consequences, and they suggest nine potential business models that depend on both directed and undirected sources of income. Semantic web technologies are what gave rise to these models. The following paper[13] discusses how internet browsers and related technologies have developed and become commonplace, making them effective instruments for various scientific problems. Present-day researchers are checking whether they can use WebGPU, WebRTC, and Internet browsers to achieve distributed cluster scale computing. The proposed work addresses a similar question by implementing a peer-to-peer cluster and testing it with matrix multiplication and Mandelbrot set generation. The experimental results demonstrate that one can

address problems with unsightly parallel structures with an efficiency of more than 75 percent. As discussed in the following paper [14], modern mobile devices like tablets and smartphones have fully functional browsers that can access rich media and web content. HTML5 introduced a platform that has made the browser more appealing to application developers. The speed gap between native applications, often written in C and C++, and web apps that incorporate web-based technologies is increasing as JavaScript engines (HTML, CSS, and JavaScript) advance. Native programs can still outperform other programs in one specific area. JavaScript is inappropriate for compute-intensive jobs such as advanced image and audio processing techniques. Specifically, the authors show how web applications can benefit from the high-performance processing capabilities of multi-core CPUs and programmable GPUs. Then, they address the standardization of this technology and its application to a mobile browser. This paper[15] discusses the use of massive open online courses (MOOCs) to educate computer science students. Students taking courses that involve programming must have access to development infrastructures that are dependable, scalable, and easy to use. The authors discovered these issues while handling the GPU programming course "Heterogeneous Parallel Programming" on Coursera. The writers investigated WebGPU, an online GPU programming environment that provides students access to an extendable and user-friendly GPU computing environment for the duration of the course. It has served as the CUDA, OpenACC, and OpenCL programming environment for extended Coursera courses, brief summer institutes, and conventional semester-long graduate and undergraduate courses. Since then, webGPU has replaced the traditional programming environment for various courses, as described in the paper. This study describes the initial, updated, and future WebGPU architectures that address the challenges of offering vital computing resources to a large, dynamic student body.

2. Important Concepts and Background

2.1. IPFS

The InterPlanetary File System (IPFS) is a peer-to-peer, decentralized file-sharing network. IPFS aims to replace the traditional centralized client-server model of the internet with a distributed network of nodes that store and share files. Instead of using a location-based address to identify files, IPFS uses a unique content-based hash, which makes the network extremely resistant to data loss and censorship.

The network distributes files via IPFS, and any node can retrieve a copy of the file from any other node. As a result, file access becomes quicker and more dependable, and there is less reliance on centralized servers, which might be subject to censorship or downtime.

IPFS applications include distributing large files, such as videos and software, creating decentralized websites, and sharing extensive data sets securely and decentralized. The IPFS community of contributors and users actively develops it as an open-source project.

2.2. WebGPU

A new low-level, high-performance web graphics and computation API is called WebGPU. It offers a mechanism for web developers to use JavaScript to access the GPU of a user's device, enabling the execution of complicated graphical and computational operations right in the browser. WebGPU is a good option for demanding applications like gaming, scientific simulations, and data visualization because it is built to deliver high performance, compatibility, and security.

2.3. WebGL

Webgl (Web Graphics Library) allows compatible web browsers to render interactive 3D and 2D graphics without the need for plug-ins. It supports GPU-accelerated rendering of intricate graphics and visualizations within web pages and is based on the OpenGL ES standard. Developers frequently utilize WebGL for creating interactive games, maps, scientific visualizations, and other graphical applications in web browsers.

2.4. Web 3.0

The World Wide Web, or Web 3.0, allows users to access documents, programs, and multimedia online through a user interface. Currently, developers are working on Web 3.0, and not everyone agrees on a single definition. However, developers will prioritize decentralized applications and use blockchain-based technology extensively in Web 3.0. Additionally, developers will utilize artificial intelligence and machine learning to create a more intelligent and responsive network.

3. Proposed System

3.1. Description of the proposed system

The authors intend to conduct an experimental analysis of heterogeneous computing for Web 3.0 applications. WebGL and WebGPU will be compared for their performance on the hosted web pages on the Web 3.0 network by creating multiple nodes. The authors will optimize these nodes, which have different computational capabilities, for the applications that work on different types of nodes.

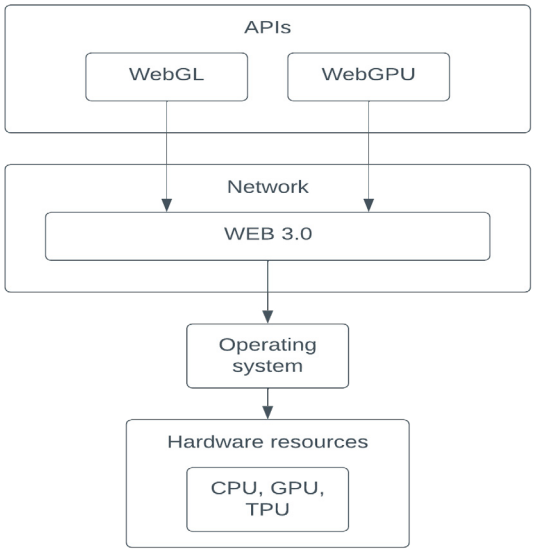


Fig. 1. Proposed System.

3.2. Target users and applications

The proposed approach will be helpful for Small and medium enterprises in developing countries focusing on Metaverse, 3D Graphics, and Manufacturing industries for Medical Devices and Equipment. Technical industries for software manufacturers and diversified technology, products, and services will also find the proposed helpful approach.

3.3. Advantages and applications of the proposed system

Cross-browser and cross-platform compatibility ensures the application runs smoothly on various browsers and operating systems. Hardware-accelerated 3D graphics enhance the visual experience by utilizing the power of the device's hardware. The browser supports parallelism, enabling the execution of multiple tasks simultaneously. It enables faster and more power-efficient performance, which allows users to access these advanced features on affordable and

widely accessible devices. The personalized web surfing experience tailors the content and recommendations according to the user's preferences and browsing habits. Data is stored on distributed nodes, providing uninterrupted services even during system failures. It brings value to virtual items by allowing users to interact with and utilize them within the browser environment. Real-time interactive 3D graphics offer an immersive and responsive experience to users. Developers can create applications that can run consistently on different hardware configurations

3.4. Requirements specification

3.4.1. Functional requirements

- The system should be able to display the content across all devices with efficiency consistently.
- The approach should include the capability to measure and compare the performance of WebGL and WebGPU APIs, involving benchmarking tests to compare rendering speed, frame rate, and memory usage.
- The approach should involve implementing WebGL and WebGPU APIs on a Web 3.0 platform using IPFS.
- Compatibility testing should ensure that the WebGL and WebGPU implementations are compatible with various web browsers, devices, and operating systems.
- The approach should involve integrating IPFS for data storage and retrieval to include storing and serving large data sets related to the performance testing and measurement of the APIs.
- A user interface should allow users to interact and visualize the performance testing and measurement results, including graphs, charts, and other visualizations.

3.4.2. Non-functional requirements

- Performance should exude consistency across all APIs.
- Different types of devices with varying hardware capabilities (Devices with GPU and without GPU) must provide users with usable performance.
- Maximum access time and throughput of Web3 Apps should be achieved while ensuring lower latency than traditional platforms.
- The page should always remain up and running, regardless of any circumstances, to guarantee portability and reliability.
- The proposed approach Implements exception handling capabilities.
- On-premises and cloud deployments of HPC servers are available options, offering on-demand access to data for enhanced workflow, depending on the organization's needs and capabilities.

3.5. System design

When a user launches a WebGPU /WebGL application hosted by a Web3.0 [browser], the operating system's application drivers improve the performance of the graphics and other essential utilities. The application module created by programmers using HTML is produced in the image above using WebGL, and then OpenGL drivers are used to giving 3D graphic rendering APIs[6] as shown in Fig 2. Fig. 3 [7] shows the webGPU based application running on the browser utlizing the GPU's.

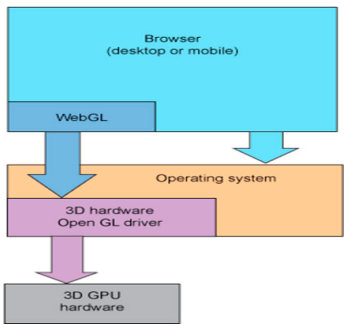


Fig. 2. WebGL application on the browser.

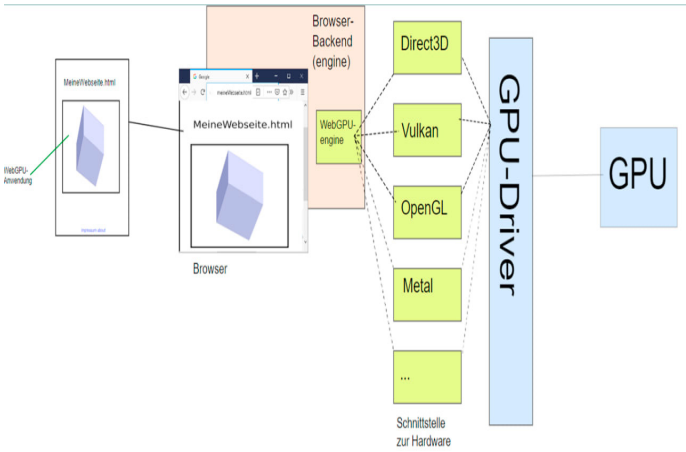


Fig. 3. WebGPU based application on the browser.

4. Implementation

The work proposed here uses WebGL and WebGPU libraries, which we store on IPFS (InterPlanetary File System). We executed the hosted application using Chrome Canary, the only browser that supports WebGPU apps. To compare the performance of the two APIs, we utilized the built-in frame rate tool in Chrome.

The method utilized the relevant WebGPU and WebGL packages to build JavaScript code. IPFS, a distributed file system that facilitates efficient and decentralized file sharing, hosted the created applications. We chose IPFS due to its ability to run web apps decentralized from centralized servers.

The hosted applications were executed using the experimental version of the Chrome browser, Chrome Canary, which supports WebGPU. We chose Chrome Canary because it's the only browser that works with WebGPU apps.

The included frame rate tool in Chrome was utilized to assess the performance of the two APIs. This tool provides real-time information on the application's frame rate and other performance indicators. By comparing the results of the two APIs using this utility, the relative performance of the two APIs under different conditions was determined.

The proposed approach allowed us to test the WebGL and WebGPU performance in a decentralized environment using IPFS and run the built applications on Chrome Canary.

4.1. Proposed methodology

4.1.1. Setting Up the Environment

We utilized Chrome Canary as our web browser, Windows 11 as our operating system and a text editor to write the code for our WebGPU and WebGL applications in our development setup. To enable support for WebGPU in Chrome Canary, we had to enable the `chrome://flags/#allow-unsafe-webGPU` flag, which is still an experimental API.

We used HTML, CSS, and JavaScript along with the WebGPU and WebGL APIs to create our applications and a range of demos. We ensured accuracy by stopping all other background operations while assessing the applications.

We built the WebGPU and WebGL apps with the exact specifications to minimize the number of performance-affecting aspects. The revolving cube example was implemented in both WebGL and WebGPU, and we made the cube size, texture, and color scheme identical. Likewise, we kept the particle size, quantity, and movement speed constant for both WebGL and WebGPU applications. We tested our applications locally using a web server.

4.1.2. Developing the Application

We created a cube model for the spinning cube application using the "WebGLGeometry" class, and then we added a texture to it using the "WebGLTexture" class. Using the "requestAnimationFrame" function, we animated the cube by causing it to rotate around its x, y, and z axes. Using the "WebGPURenderPassEncoder" class, we rendered the cube mesh and applied the same texture to the WebGPU version of the software.

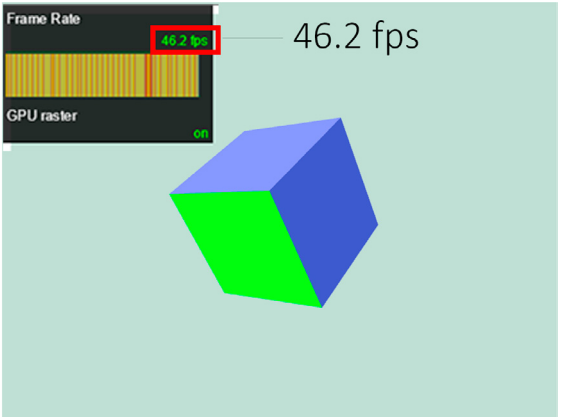


Fig. 4. Frame rate recorded after running 15 instances of WebGL application.

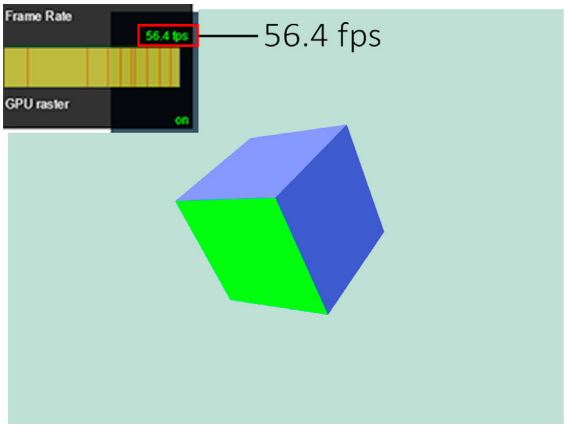


Fig. 5. Frame rate recorded after running 15 instances of WebGPU application.

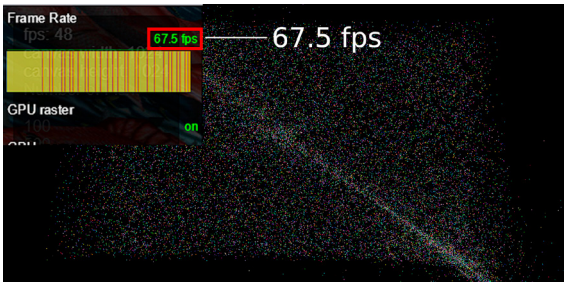


Fig. 6. Frame rate recorded after running 15 instances of WebGL program containing 50000 particles each.

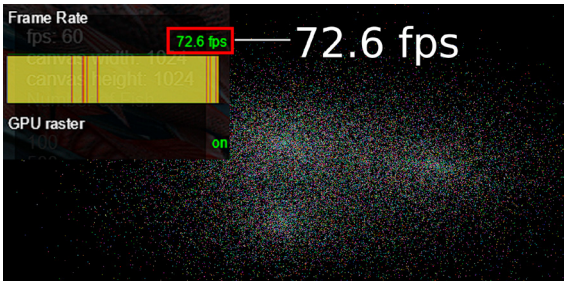


Fig. 7. Frame rate recorded after running 15 instances of WebGPU program containing 50000 particles each.

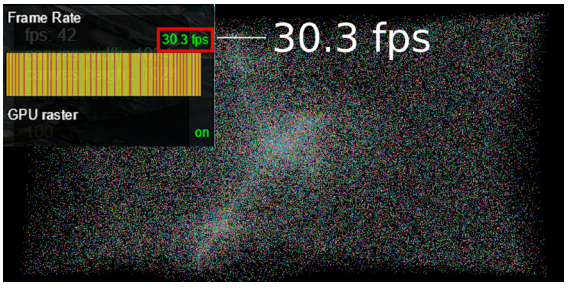


Fig. 8. Frame rate recorded after running 40 instances of WebGL program containing 50000 particles each.

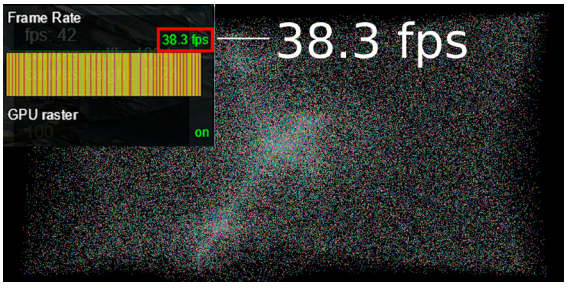


Fig. 9. Frame rate recorded after running 40 instances of WebGPU program containing 50000 particles each.

Using the ”WebGLBuffer” class, we created a random collection of particles for the particle scatter plot application. We then updated their positions over time with the ”requestAnimationFrame” function. The particles were given a color gradient according to their distance and speed from the scatter plot’s center. We calculated the new particle coordinates and applied the same color gradient for the WebGPU version of the application using the ”WebGPUComputePassEncoder” class.

Several obstacles specific to each API were faced during the development process. For instance, creating more code was necessary to do the same tasks with WebGPU due to its stricter structure and syntax than with WebGL, but this also increased efficiency and allowed us to leverage more recent hardware.

We used various programming techniques to make the code understandable and straightforward, such as inheritance, encapsulation, and modularisation. Ultimately, using WebGPU and WebGL programs, we created two visually appealing applications that demonstrated the capabilities of both APIs while upholding the same performance standards.

4.1.3. Hosting the Applications on IPFS

IPFS (InterPlanetary File System), a peer-to-peer network for storing and exchanging files, hosted our applications. We installed the IPFS desktop client on our PCs and added our programs to create an IPFS node.

We used the IPFS desktop client to initially create a content hash for the application before we hosted the program on IPFS. We added the application to our local IPFS node via the command line interface, and the IPFS desktop client automatically propagated the program to other nodes in the network.

We entered the content hash for each application in the browser's URL bar to access the applications in a web browser. The programs could be tested in the browser once they were loaded from the IPFS network.

We stored two webGPU and webGL samples on IPFS for our performance testing: a rotating cube and a particle scatter plot. We verified that other IPFS network users could access these programs using the IPFS desktop client.

We used IPFS as the identical hosting solution for both types of apps to ensure a precise comparison of the webGPU and webGL APIs. To spot any performance variations between the two APIs, we closely tracked the resources each program consumed, such as CPU and GPU consumption. A more accurate comparison of webGPU and webGL performance was also achieved by averaging the outcomes of each application's numerous executions. This was done to guarantee consistent results.

4.1.4. Collecting performance data and analysing performance data

We gathered performance statistics while running two applications—one with rotating cubes and the other with scattered particles moving—to compare the performance of WebGPU and WebGL APIs. The same settings and requirements were used to construct both applications, including the cube size, texture, colors, particle size, movement speed, and quantity. To reduce any performance-affecting factors, this was done.

We collected the performance data on a Windows 11 operating system, using the Chrome Canary browser with the `chrome://flags/#enable-unsafe-webgpu` flag enabled. To host the webpages, we used the IPFS desktop client. All other background applications were closed during the testing process to ensure that the system resources were fully dedicated to the applications.

To assess the efficiency of the WebGL and WebGPU APIs, we ran various instances of the applications with multiple quantities of particles. We used 15, 20, and 40 cases for the particle application and only 15 for the cube application. We calculated the frames per second (fps) for each case using Chrome's built-in framerate tool.

Our performance statistics demonstrated that the WebGPU API consistently outperformed the WebGL API. For the particle application, the WebGPU frame rates for 15, 20, and 40 instances were 72.7, 62.3, and 38.3, respectively. In contrast, the frame rates were 67.5, 50.2, and 30 for the same number of cases using WebGL. The cube application's frame rate on WebGPU was 56.4, whereas it was 46.2 on WebGL.

4.2. Experimental settings

All experiments are compiled and tested on a Windows machine(CPU: Intel core i5-10210U, GPU: Nvidia MX250 2GB, Memory: 8GB 2400 MHz)

4.3. Results

The WebGPU API would be a better choice for developing high-performance web applications that require graphics-intensive processes, as suggested by our findings. Remembering that performance can change based on the particular hardware and software settings is crucial.

We didn't use any screen capture software to record the results because it would consume system resources and reduce performance. Hence, we recorded the results using an external camera. These screenshots are obtained from the video recording and are shown in Fig.4, Fig.5, Fig.6, Fig.7, Fig.8 and Fig.9.

Table 1. Comparison of WebGPU and WebGL Performance Results

Application	WebGPU (fps)	WebGL (fps)
Particles (15 instances)	72.7	67.5
Particles (20 instances)	62.3	50.2
Particles (40 instances)	38.3	30
Cube (15 instances)	56.4	46.2

5. Conclusion

The comparison of WebGL and WebGPU APIs in Web 3.0 has produced several significant findings. Further, this study has discovered that WebGPU outperformed WebGL, with noticeably higher frame rates for the same apps. This is probably because WebGPU has better hardware utilization and performance enhancements. This study has also conducted performance tests in a stable and reliable environment by implementing the WebGPU and WebGL applications using HTML, CSS, and JavaScript, hosting them on IPFS, and running them on Chrome Canary. Further, the performance differences were solely due to the APIs and not any other confounding factors. The identical parameters between the applications, such as cube size, texture, colors, particle size, speed, and number are maintained.

The proposed reliable methodology has been enabled to obtain precise performance data for WebGPU and WebGL applications using various samples, such as rotating cubes, scattering particles, and similar applications. The data we collected showed that WebGPU consistently outperformed WebGL regarding frame rates. The significance of choosing the correct API for Web 3.0 apps is emphasized by our analysis, with WebGPU outperforming WebGL in terms of performance. This can significantly impact how users interact with a Web 3.0 application and ultimately determine its success.

Acknowledgment

The authors would like to thank KLE Technological University for supporting this research.

References

[1] Almeida, F., Santos, J. D., Monteiro, J. A. (2013). E-Commerce business models with the context of Web 3.0 paradigm. International Journal of Advanced Information Technology, 3(6), 1-12.

[2] Data growth and Web 3.0. (2017). 5 main features of Web 3.0. Retrieved March 10, 2018,

[3] Hussain, F. (2012). E-learning 3.0=E-learning 2.0 + Web 3.0?. Proceedings of the IADIS International Conference on Cognition and Exploratory Learning in Digital Age.

[4] Ryan Browne. 2022. The luna cryptocurrency has been resurrected after its Dollar 40 billion collapse. It's already crashing. https://www.cnbc.com/2022/05/30/terra2point0-new-luna-cryptocurrency-is-already-crashing.html

[5] Nupur Choudhury. 2014. World Wide Web and its journey from Web 1.0 to Web 4.0. International Journal of Computer Science and Information Technologies.

[6] Shumo Chu and Sophia Wang. 2018. The curses of blockchain decentralization. arXiv:1810.02937 (2018)

[7] Ru-Xi Ding, Iván Palomares, Xueqing Wang, Guo-Rui Yang, Bingsheng Liu, Yucheng Dong, Enrique Herrera-Viedma, and Francisco Herrera. 2020. LargeScale decision-making: Characterization, taxonomy, challenges and future directions from an Artificial Intelligence and applications perspective. Information Fusion 59 (2020)

[8] GeeksforGeeks. 2021. How Web 3.0 is going to impact the digital world? https:// www.geeksforgeeks.org/how-web-3-0-is-going-to-impact-the-digital-world/

[9]Introducing WebGL - https://developer.ibm.com/tutorials/wa-webgl1/

[10] WebGPU - 3D im Browser - https://www.peter-strohm.de/webgpu/

[11] Potluri, Rajasekhara Mouly Vajjhala, Narasimha. (2018). A Study on Application of Web 3.0 Technologies in Small and Medium Enterprises of India. The Journal of Asian Finance, Economics and Business. 5. 73-79. 10.13106/jafeb.2018.vol5.no2.73.

[12] Gan, Wensheng, Zhenqiang Ye, Shicheng Wan, and Philip S. Yu. "Web 3.0: The Future of Internet." arXiv preprint arXiv:2304.06032 (2023).

[13] Bachelor Thesis:Bachelor Of Science Programme in Computing Science of Abdulsalam Aldahir accessed from https://www.diva-portal.org/smash/get/diva2:1674447/FULLTEXT01.pdf

[14] Jeon, Won Brutch, Tasneem Gibbs, Simon. (2012). WebCL for Hardware-Accelerated Web Applications. Tizen Developer Conference.

[15] A. Dakkak, C. Pearson and W. -M. Hwu, "WebGPU: A Scalable Online Development Platform for GPU Programming Courses," 2016 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), Chicago, IL, USA, 2016, pp. 942-949, doi: 10.1109/IPDPSW.2016.63.